

AWS EC2 Based Student Registration Portal

1)Executive Summary

This project demonstrates the deployment of a Student Registration Portal on AWS Cloud using Amazon EC2, Apache, PHP and MariaDB. The application allows students to submit registration information through a web interface and stores the data in a relational database.

2)Problem Statement

3)Project Objectives

- Deploy a web application on AWS EC2
- Configure Apache and PHP
- Create and manage MariaDB database
- Enable frontend-backend integration
- Demonstrate cloud deployment and database connectivity

4)Technology Stack

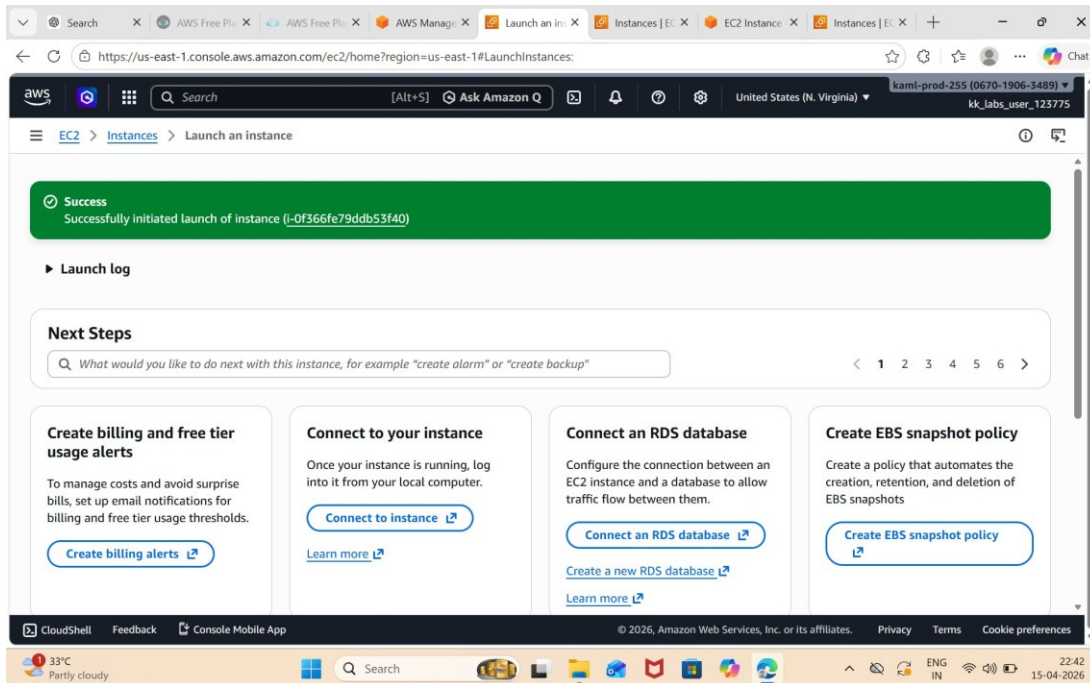
AWS EC2, Amazon Linux, Apache HTTP Server, PHP, MariaDB, HTML, CSS, SQL, Linux

5)System Architecture

The project follows a 2-Tier Architecture. Apache and PHP run on AWS EC2 as the application layer. MariaDB acts as the database layer storing student records.

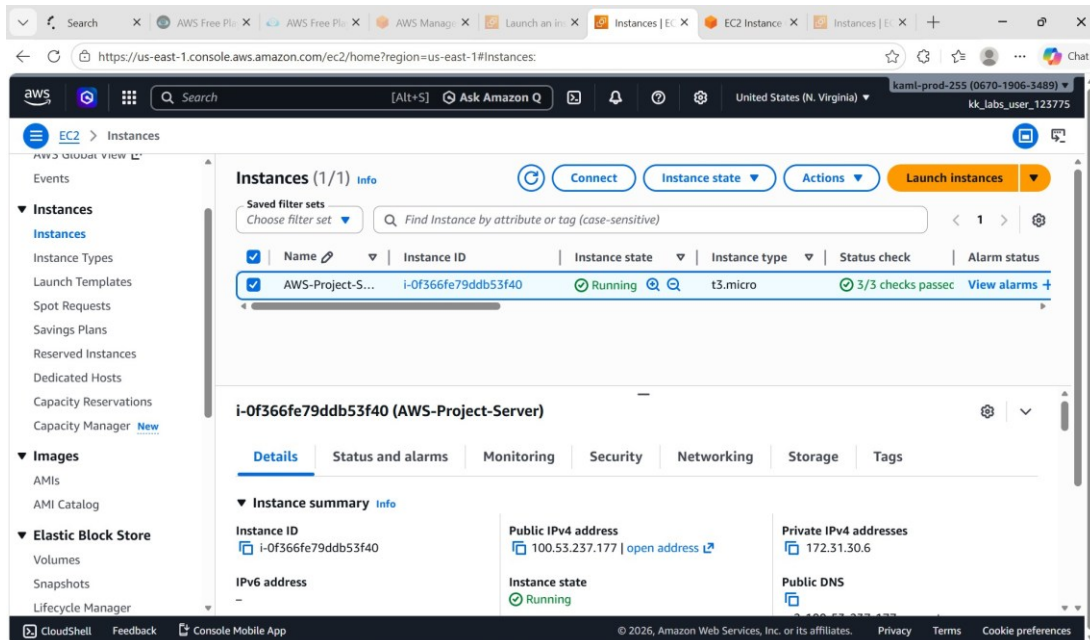
Step 1 - EC2 Launch Success

Screenshot evidence for step 1 - ec2 launch success.



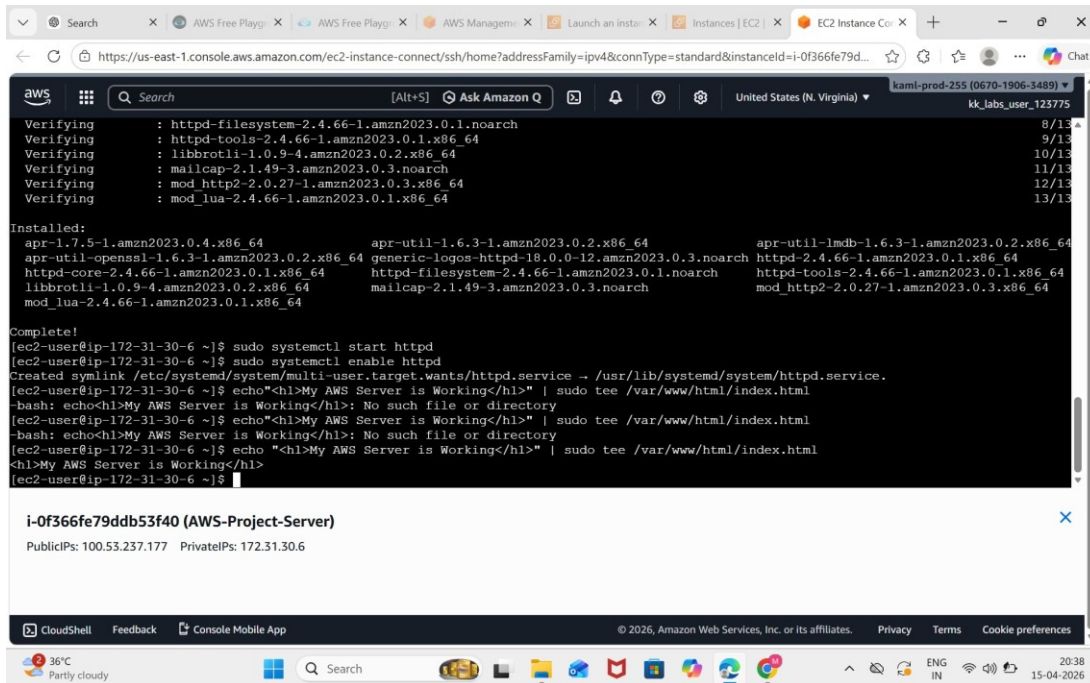
Step 2 - EC2 Instance Running

Screenshot evidence for step 2 - ec2 instance running.



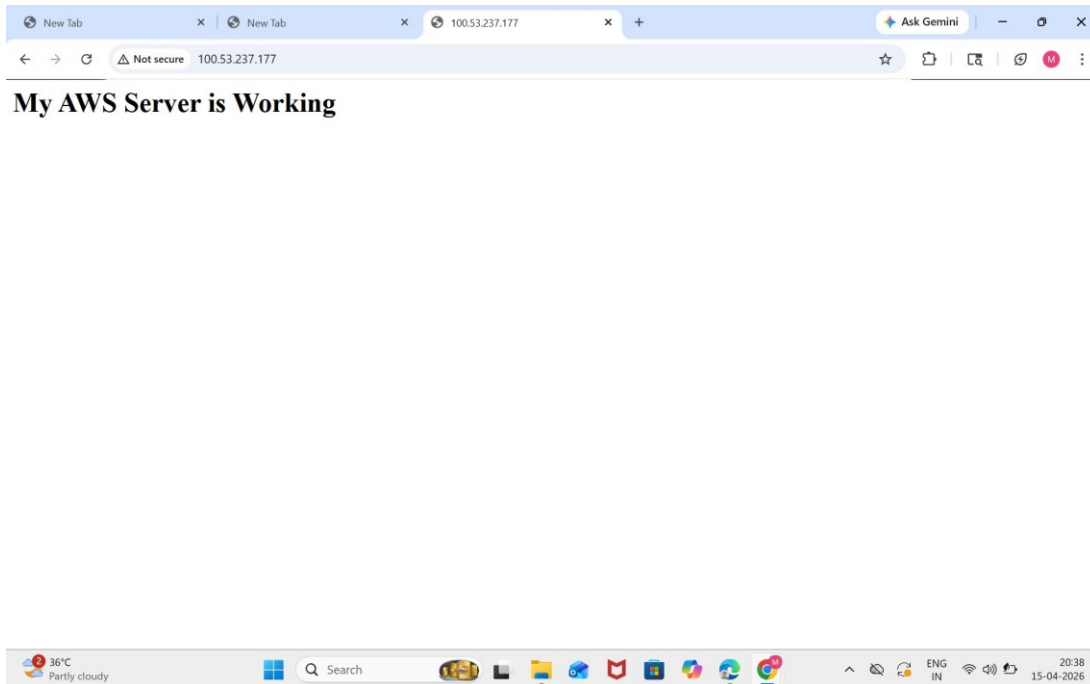
Step 3 - Apache Server Setup

Screenshot evidence for step 3 - apache server setup.



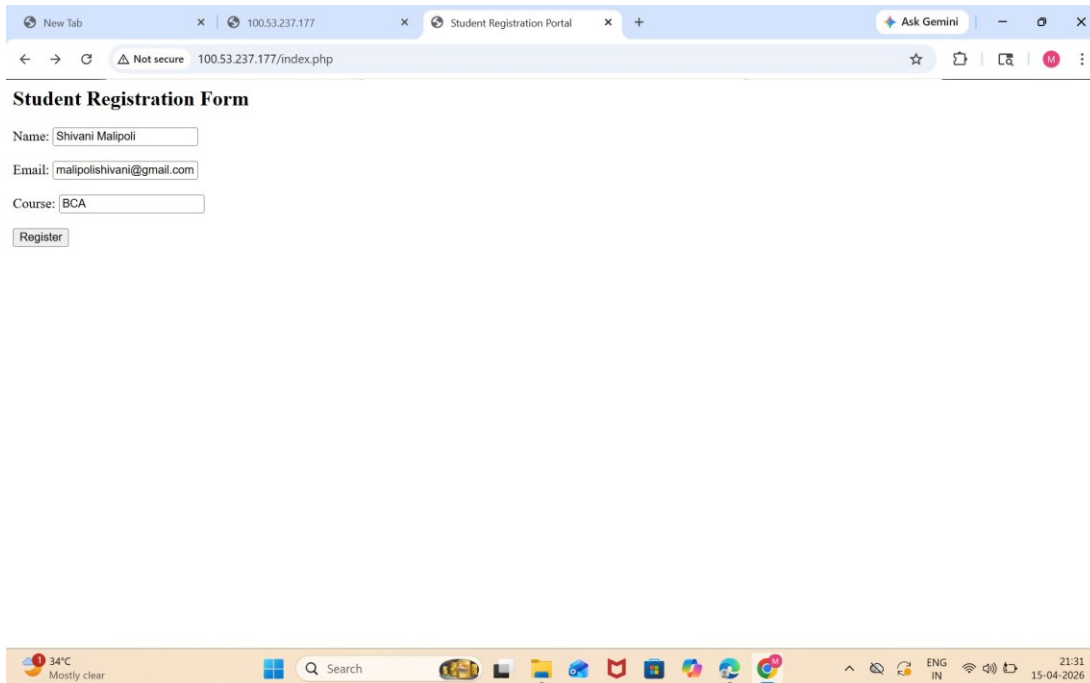
Step 4 - Apache Verification

Screenshot evidence for step 4 - apache verification.



Step 5 - Application Deployment

Screenshot evidence for step 5 - application deployment.



Step 6 - MariaDB Setup

Screenshot evidence for step 6 - MariaDB setup.

```
[ec2-user@ip-10-0-0-134 ~]$ sudo mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 15
Server version: 10.5.22-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| studentdb |
| sys |
+-----+
5 rows in set (0.002 sec)

MariaDB [(none)]> use studentdb;
Database changed
MariaDB [studentdb]> show tables;
+-----+
| Tables_in_studentdb |
+-----+
| students |
+-----+
1 row in set (0.001 sec)

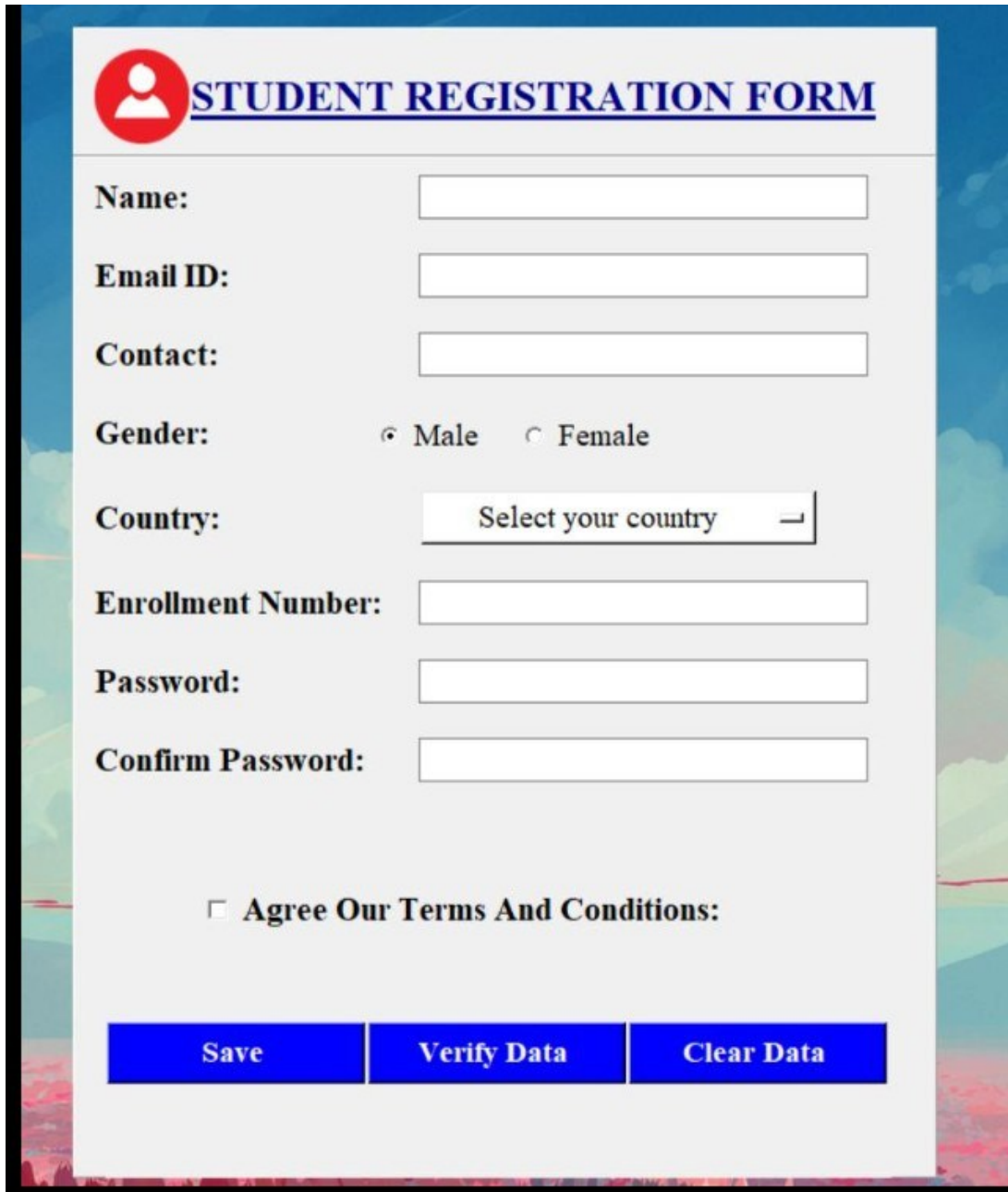
MariaDB [studentdb]> describe students;
+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+
| id | int(11) | NO | PRI | NULL | auto_increment |
| name | varchar(100) | YES | | NULL | |
| email | varchar(100) | YES | | NULL | |
| contact | varchar(15) | YES | | NULL | |
| course | varchar(50) | YES | | NULL | |
| created_at | timestamp | YES | | current_timestamp() | |
+-----+
6 rows in set (0.001 sec)

MariaDB [studentdb]> select * from students;
+-----+
| id | name | email | contact | course | created_at |
+-----+
| 1 | Shivani | shivani.mali@example.com | 9876543210 | BCA | 2025-05-15 12:45:10 |
| 2 | Rahul | rahul.kumar@example.com | 9123456780 | BSc IT | 2025-05-15 12:48:33 |
+-----+
2 rows in set (0.001 sec)

MariaDB [studentdb]>
```

Step 7 - Registration Form UI

Screenshot evidence for step 7 - registration form ui.



The screenshot displays a web form titled "STUDENT REGISTRATION FORM" with a red circular logo containing a white person icon. The form is set against a light gray background with a blue border. It includes several input fields: "Name:", "Email ID:", "Contact:", "Enrollment Number:", "Password:", and "Confirm Password:". The "Gender:" field has radio buttons for "Male" and "Female", with "Male" selected. The "Country:" field is a dropdown menu showing "Select your country". Below the input fields is a checkbox labeled "Agree Our Terms And Conditions:". At the bottom, there are three blue buttons: "Save", "Verify Data", and "Clear Data".

STUDENT REGISTRATION FORM

Name:

Email ID:

Contact:

Gender: ☒ Male ☐ Female

Country:

Enrollment Number:

Password:

Confirm Password:

☐ **Agree Our Terms And Conditions:**

Save **Verify Data** **Clear Data**

Step 8 - Database Connectivity Testing

Screenshot evidence for step 8 - database connectivity testing.

The screenshot shows a terminal window within an AWS EC2 instance. The terminal output is as follows:

```
Query OK, 0 rows affected (0.008 sec)

MariaDB [studentdb]> exit;
Bye
[ec2-user@ip-172-31-30-6 ~]$ sudo nano /var/www/html/submit.php
[ec2-user@ip-172-31-30-6 ~]$ sudo mysql
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 7
Server version: 10.5.29-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> SELECT * FROM students;
ERROR 1046 (3D000): No database selected
MariaDB [(none)]> USE studentdb;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [studentdb]> SELECT * FROM students;
Empty set (0.001 sec)

MariaDB [studentdb]>
```

Below the terminal window, a metadata box for the instance `i-Of366fe79ddb53f40 (AWS-Project-Server)` is visible, showing PublicIPs: 100.53.237.177 and PrivateIPs: 172.31.30.6. The bottom of the image shows the Windows taskbar with the date 15-04-2026 and time 21:36.

Database Verification and Backend Testing

This screenshot demonstrates the successful verification of the MariaDB database connectivity on the AWS EC2 instance. After configuring the database and deploying the Student Registration Portal, SQL commands were executed to validate the connection between the application layer and the database layer.

The student dB database was successfully selected, and SQL queries were executed to verify that the database was accessible from the server environment. The successful execution of the SELECT query confirmed that the MariaDB service was running correctly and that the application could communicate with the database without connectivity issues.

This testing phase played a crucial role in validating the backend functionality of the project. It ensured that the database configuration, user permissions, and server settings were properly established. By performing these validation checks, the reliability and stability of the Student Registration Portal were confirmed before production deployment.

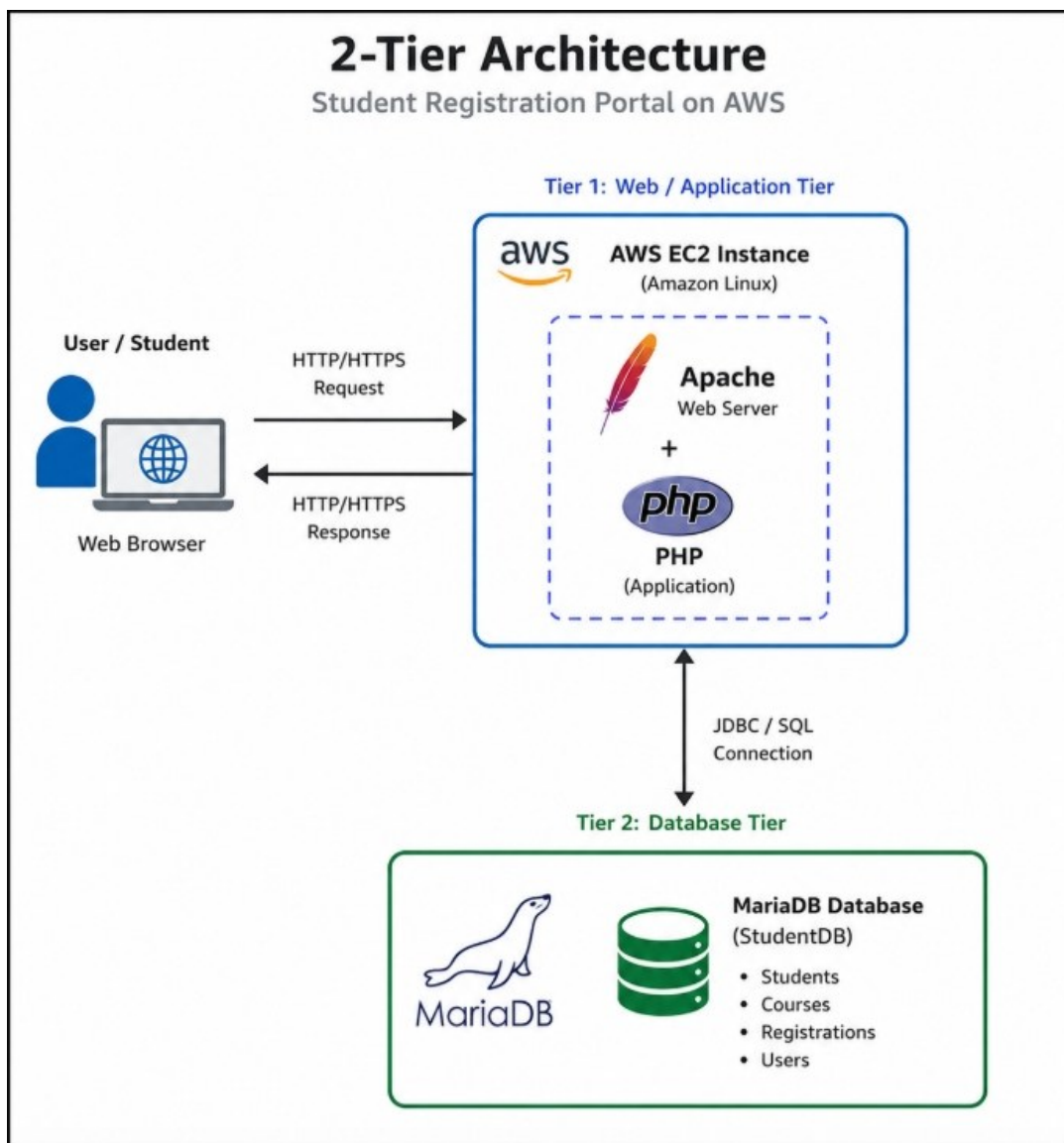
Key Outcomes:

- Verified MariaDB database accessibility.

- Confirmed successful database selection and query execution.
- Validated communication between the application and database layers.
- Ensured proper backend configuration on AWS EC2.
- Demonstrated successful database integration within the 2-Tier Architecture.

This verification process confirms that the Student Registration Portal can reliably store, retrieve, and manage student information within the cloud-hosted environment.

Step9 - Architecture Diagram



6)AWS Services Used

Amazon EC2 (Elastic Compute Cloud)

Amazon EC2 was used as the primary cloud computing service to host the Student Registration Portal application. An Amazon Linux instance was launched and configured to run the Apache Web Server, PHP application, and MariaDB database.

Purpose:

- Host the web application.
 - Execute backend PHP scripts.
 - Manage database operations.
 - Provide public access through a public IP address.
-

Amazon Linux

Amazon Linux was used as the operating system for the EC2 instance. It provided a secure and stable environment for application deployment and server administration.

Purpose:

- Server management.
 - Software installation.
 - Service configuration.
 - Linux command execution.
-

Security Groups

AWS Security Groups were configured as virtual firewalls to control inbound and outbound traffic to the EC2 instance.

Configured Rules:

- Port 22 (SSH) – Remote server access.
- Port 80 (HTTP) – Web application access.

Purpose:

- Secure the EC2 instance.

- Control network traffic.
 - Restrict unauthorized access.
-

Public IP Address

A public IP address was assigned to the EC2 instance to allow users to access the Student Registration Portal through a web browser.

Purpose:

- Enable external access.
 - Test application deployment.
 - Verify web server functionality.
-

EC2 Instance Connect

EC2 Instance Connect was used to access the Linux terminal directly from the AWS Management Console without requiring third-party SSH tools.

Purpose:

- Server administration.
 - Software installation.
 - Application deployment.
 - Database management.
-

AWS Cloud Infrastructure Benefits

- On-demand resource provisioning.
- High availability.
- Cost-effective hosting.
- Scalability for future enhancements.
- Real-world cloud deployment experience.

7)Database Design

Table: students

Fields: id, name, email, contact, course, created_at

Primary Key: id

8)Key Code Snippets

Database Connection (PHP)

The following code establishes a connection between the PHP application and the MariaDB database hosted on the AWS EC2 instance.

```
<?php

$conn = new mysqli("localhost", "root", "password", "studentdb");

if ($conn->connect error) {

    die ("Connection failed: ". $conn->connect error);

}

?>
```

Student Table Creation (SQL)

The students table stores all registration information submitted through the web application.

```
CREATE TABLE students (

    id INT AUTO_INCREMENT PRIMARY KEY,

    name VARCHAR(100),

    email VARCHAR(100),

    contact VARCHAR(15),

    course VARCHAR (50),

    created at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

);
```

Student Registration Data Insertion

The following SQL query is executed through PHP when a student submits the registration form.

```
$sql = "INSERT INTO students  
(name, email, contact, course)  
VALUES  
('$name', '$email', '$contact', '$course')";
```

Purpose of the Code

- Establishes secure communication between PHP and MariaDB.
- Stores student registration records in a structured database.
- Demonstrates backend processing and database integration.
- Validates successful interaction between the application layer and database layer.
- Supports the implementation of the 2-Tier Architecture deployed on AWS EC2.

9)Application Workflow

User opens the registration form, enters details, submits the form, PHP processes the request, and data is stored in MariaDB.

10)Implementation Steps

1. Launch EC2
2. Configure Security Group
3. Install Apache
4. Install PHP
5. Install MariaDB
6. Create Database
7. Deploy Application
8. Test Connectivity
9. Verify Data Storage

11)Testing Results

Application successfully served through EC2 public IP. Student records were inserted and verified in MariaDB.

12)Challenges and Solutions

Resolved package installation issues, database connection configuration, Apache service configuration and deployment verification.

13)Key Learning Outcomes

AWS EC2 administration, Linux commands, Apache configuration, PHP integration, MariaDB management, SQL operations and cloud deployment.

14)Future Enhancements

User authentication, HTTPS, AWS RDS migration, responsive UI, email notifications, load balancing.

15)GitHub Repository Structure

README.md
screenshots/
source-code/
database/
docs/

16)Interview Talking Points

Explain EC2 deployment, Apache setup, PHP-MariaDB integration, architecture design, database testing and cloud hosting workflow.

17)Business Impact

Eliminates manual student registration processes.

Centralizes student information management.

Provides cloud-based accessibility from anywhere.

Demonstrates scalable deployment using AWS infrastructure.

Reduces administrative effort and improves data accuracy.

18)Security Features Implemented

AWS Security Groups configured for controlled access.

MariaDB database hosted within the EC2 environment.

Server-side processing using PHP.

Data stored in a structured relational database.

Separation between application and database layers using 2-Tier Architecture.

19)Why This Project Stands Out

End-to-end cloud deployment on AWS EC2.

Real-world database integration using MariaDB.

Linux server administration experience.

Apache web server configuration.

PHP backend development.

SQL database design and query execution.

Application deployment and troubleshooting.

Understanding of cloud architecture and hosting.

20)Conclusion

This project demonstrates practical cloud deployment skills and full-stack integration using AWS, making it a strong portfolio project for fresher software, cloud and web development roles.